Tutorial #2 – GitHub & AWS CodeBuild

Resumen	Tutorial - GitHub & AWS CodeBuild
URL	devops-GitHub-AWS-Codebuild
Categoría	devops
Tags	devops
Estado	Completado
Autor(es)	Equipo de tutores curso de DevOps Mario José Villamizar Cano

Introducción

Nombre:	**Tutorial #2 - GitHub & AWS CodeBuild**
Duración:	60 minutos
Profesor responsable	Mario Villamizar
Pre-Requisitos:	Python + Flask + Git + AWS IaaS

Objetivos

Al finalizar el tutorial el estudiante estará en capacidad de:

- Aprovisionar repositorios sobre GitHub y realizar compilaciones de código a través de AWS CodeBuild

Requisitos para desarrollar el tutorial

En particular se utilizarán los siguientes recursos:

1. Cuenta de AWS
2. Cuenta GitHub
3. Conocimientos en Python, Flask, Git y la consola de administración web de AWS

Repositorio

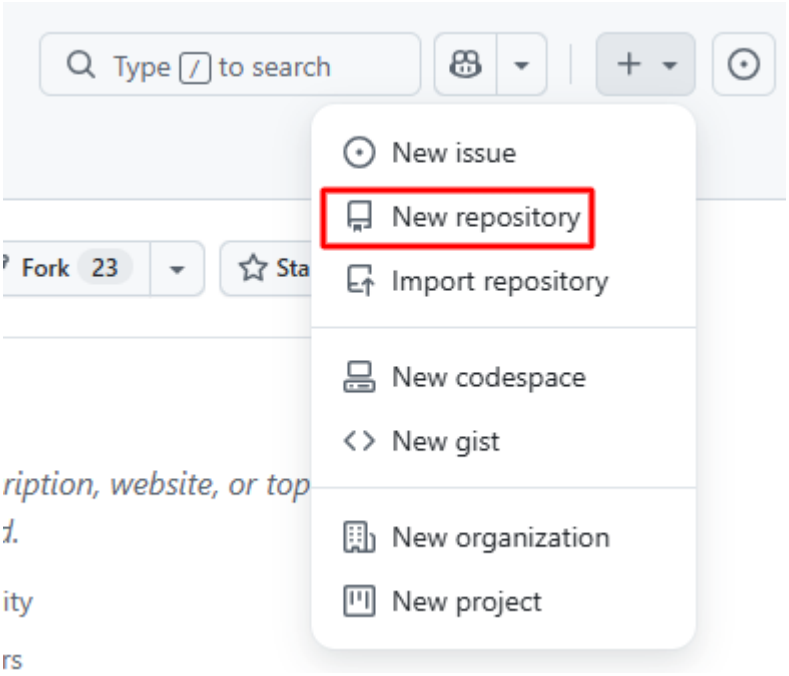
El código de la aplicación lo encuentra en el siguiente repositorio de GitHub:
https://github.com/WeimarG/taller-pipeline-beanstalk.git
La aplicación es una pequeña API en Python y Flask con dos endpoints.

Endpoint	**/**
Descripción	Lista la información de todos los héroes registrados en plataforma.
Método	GET
Retorno	**application/json**, con todos los héroes registrados en plataforma.
Parámetros del Endpoint	Ninguno

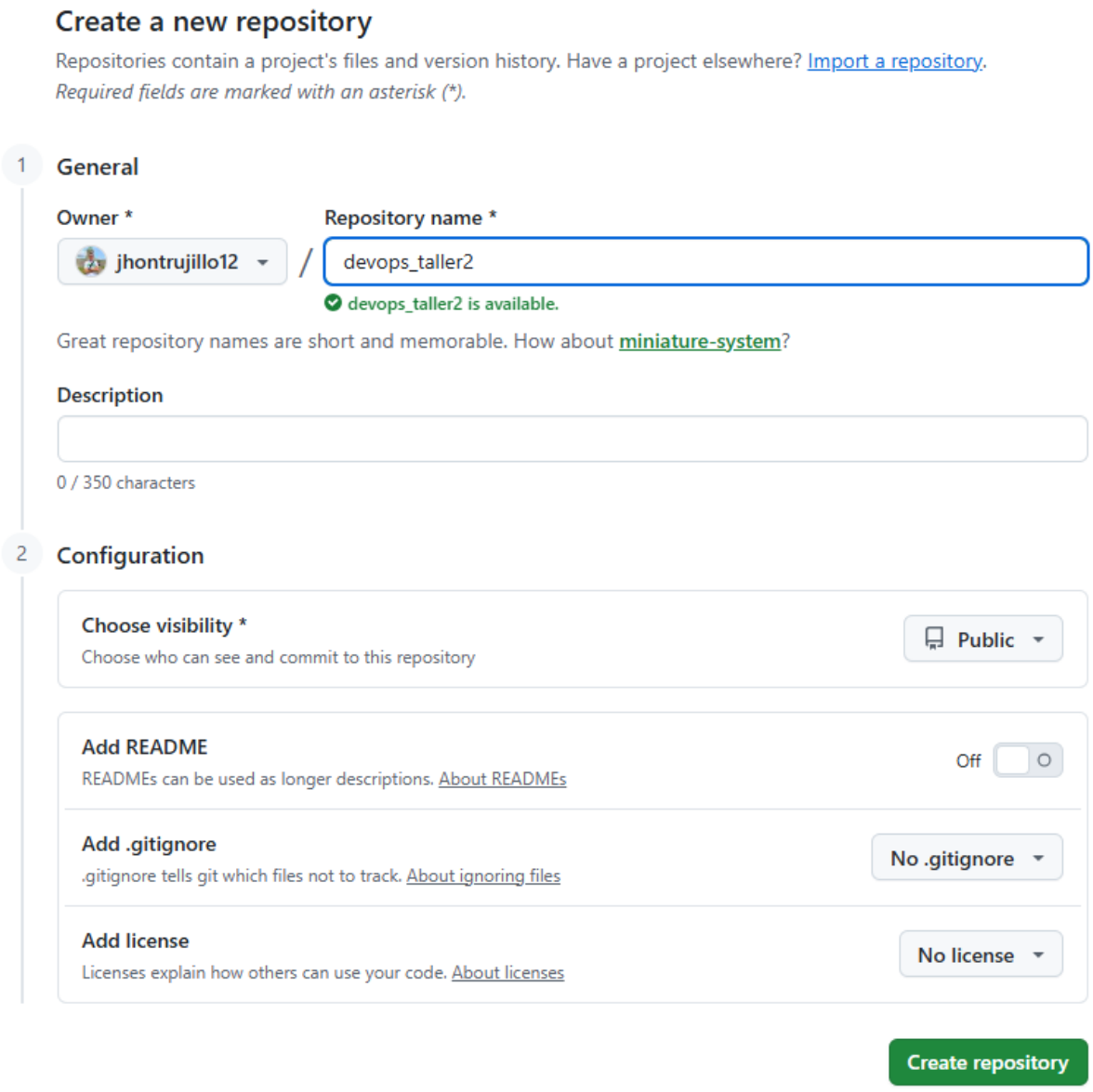
Endpoint	**/<int:id_heroe>**
Descripción	Permite recuperar la información de un héroe registrado en plataforma.
Método	GET
Retorno	**application/json**. Con un diccionario que representa la información de un héroe.
Parámetros del Endpoint	Ninguno.

Actividad 1 – Creación de repositorio en GitHub

Inicie sesión en su cuenta de GitHub y en la barra superior de GitHub seleccione el botón **(+)** y de clic en la opción “New repository”



En la nueva ventana asigne un nombre a su repositorio y seleccione si el repositorio será público o privado, para este caso seleccione la opción Público y finalice dando clic en el botón “*Create repository*”.

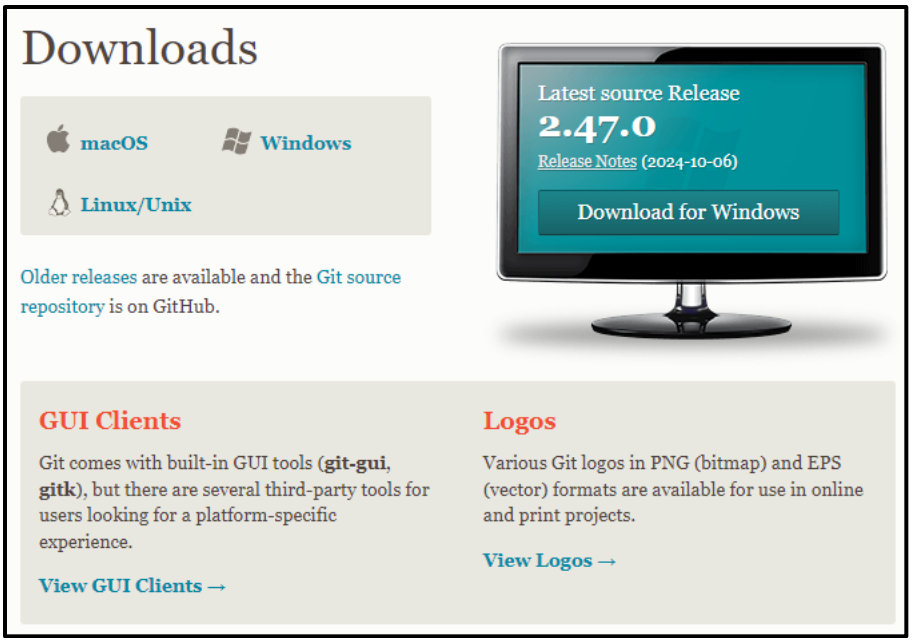


Actividad 2 – Configuración git en su máquina local

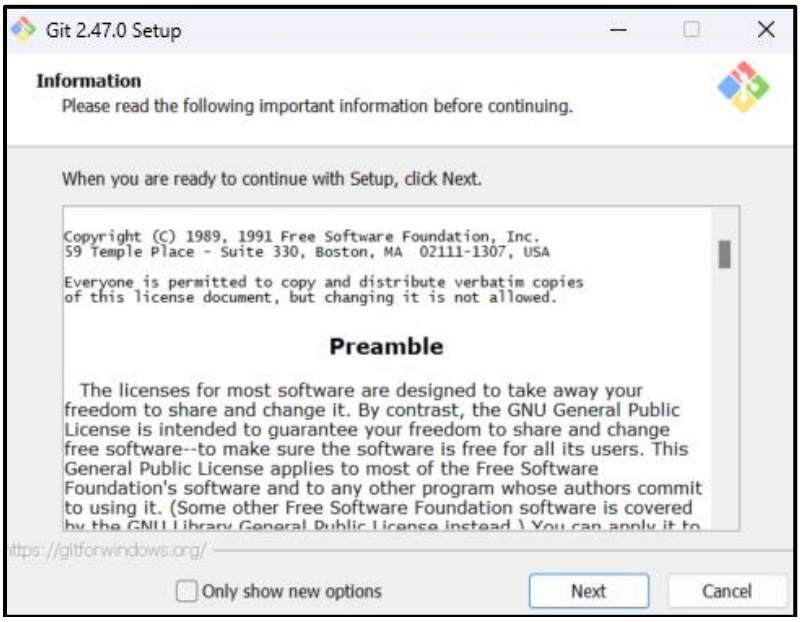
Puede descargar Git a través de la siguiente URL:

https://git-scm.com/download

Dentro de las opciones que ofrece Git, puede instalarlo en sistemas operativos Linux, macOS y Windows. Para este tutorial se procederá con la instalación en un sistema operativo Windows



Después de la descarga ejecute el instalador y seleccione la configuración según sus necesidades.



Puede validar la versión de Git el siguiente comando:

```
C:\Windows\System32>git --version
git version 2.53.0.windows.2

C:\Windows\System32>
```

Actividad 3 – Cargue de aplicación al repositorio.

Clone el repositorio en su máquina local e ingrese al folder.

> **git clone https://github.com/WeimarG/taller-pipeline-beanstalk.git tutorial-2-devops**





Regrese nuevamente a la consola de su equipo local, ingrese dentro del directorio del repositorio de la aplicación y ejecute el comando:

> **cd tutorial-2-devops**

> **git remote remove origin**

> **git remote add origin "URL to Your GitHub Repository Link HTTPS"**

Donde **"URL to Your GitHub Repository Link HTTPS"** corresponde a la url del repositorio creado en GitHub. A manera de ejemplo el comando sería:

> ***git remote add origin https://github.com/WeimarG/Devops_tutorial2.git***

Prepare todos los archivos de la aplicación y realice un push de estos a su repositorio de GitHub con el comando:

> ***git add .***

> ***git commit -m "First commit"***

> ***git push -u origin master***

> ***git branch***

```
AzureAD+JhonJairoMorenoTruji@DEVSECOPS MINGW64 /d/Repos
$ git clone https://github.com/WeimarG/taller-pipeline-beanstalk.git tutorial-2-devops
Cloning into 'tutorial-2-devops'...
remote: Enumerating objects: 9, done.
remote: Total 9 (delta 0), reused 0 (delta 0), pack-reused 9 (from 1)
Receiving objects: 100% (9/9), done.

AzureAD+JhonJairoMorenoTruji@DEVSECOPS MINGW64 /d/Repos
$ cd tutorial-2-devops/

AzureAD+JhonJairoMorenoTruji@DEVSECOPS MINGW64 /d/Repos/tutorial-2-devops (master)
$ git remote remove origin

AzureAD+JhonJairoMorenoTruji@DEVSECOPS MINGW64 /d/Repos/tutorial-2-devops (master)
$ git remote add origin https://github.com/jhontrujillo12/devops_taller2.git

AzureAD+JhonJairoMorenoTruji@DEVSECOPS MINGW64 /d/Repos/tutorial-2-devops (master)
$ git add .

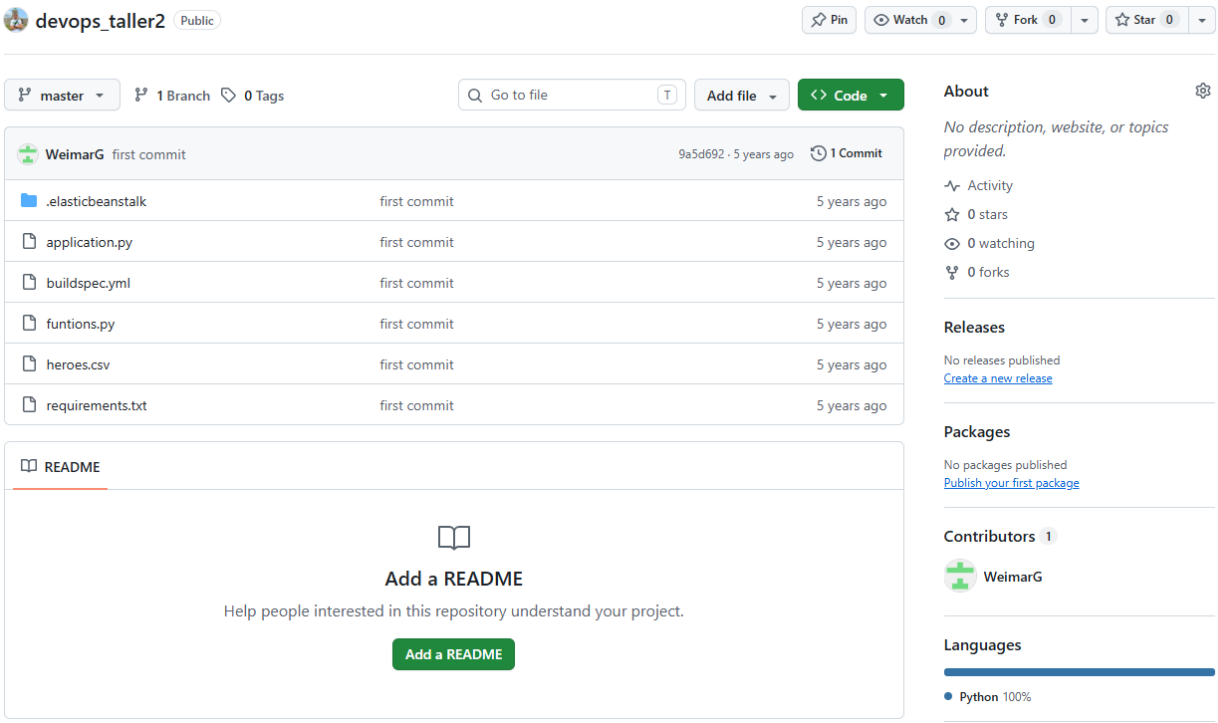
AzureAD+JhonJairoMorenoTruji@DEVSECOPS MINGW64 /d/Repos/tutorial-2-devops (master)
$ git commit -m "First commit"
On branch master
nothing to commit, working tree clean

AzureAD+JhonJairoMorenoTruji@DEVSECOPS MINGW64 /d/Repos/tutorial-2-devops (master)
$ git push -u origin master
info: please complete authentication in your browser...
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 12 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (9/9), 1.63 KiB | 1.63 MiB/s, done.
Total 9 (delta 0), reused 9 (delta 0), pack-reused 0 (from 0)
To https://github.com/jhontrujillo12/devops_taller2.git
 * [new branch]      master -> master
branch 'master' set up to track 'origin/master'.

AzureAD+JhonJairoMorenoTruji@DEVSECOPS MINGW64 /d/Repos/tutorial-2-devops (master)
$ git branch
* master

AzureAD+JhonJairoMorenoTruji@DEVSECOPS MINGW64 /d/Repos/tutorial-2-devops (master)
$
```

A continuación, ingrese nuevamente a su cuenta de GitHub desde el navegador para validar que los archivos se hayan cargado correctamente al repositorio.



AWS CodeBuild

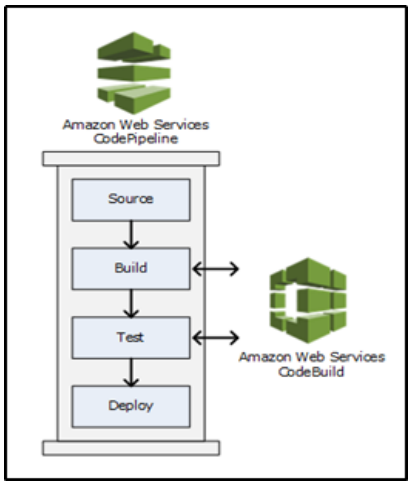
AWS CodeBuild es un servicio de compilación de código administrado por AWS. Este servicio permite realizar test unitarios y genera artefactos para su implementación en producción.

Con AWS CodeBuild puede realizar pruebas sobre las diferentes versiones de su aplicación, esto se debe a que se puede integrar con diferentes herramientas como GitHub, AWS CodeCommit, S3, entre otros.

AWS CodeBuild no necesita la implementación ni administración de infraestructura como servidores o grupos de seguridad ya que es un servicio serverless donde lo único que debe hacer es cargar el código de su aplicación en un repositorio y AWS CodeBuild se conectará para realizar la compilación.

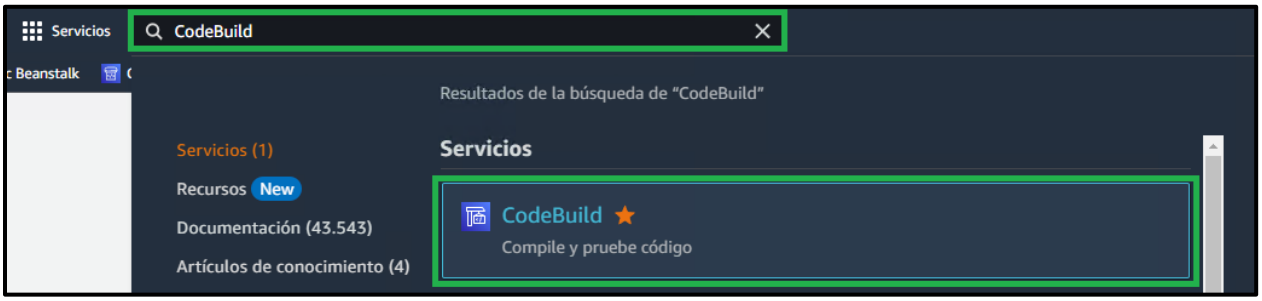
CodeBuild es compatible con los lenguajes de programación más conocidos por lo que no debe preocuparse por utilizar un lenguaje en específico.

AWS CodeBuild permite realizar integraciones con herramientas como CodePipeline para ser parte de una canalización y posteriormente realizar el despliegue de la aplicación en un ambiente de producción.

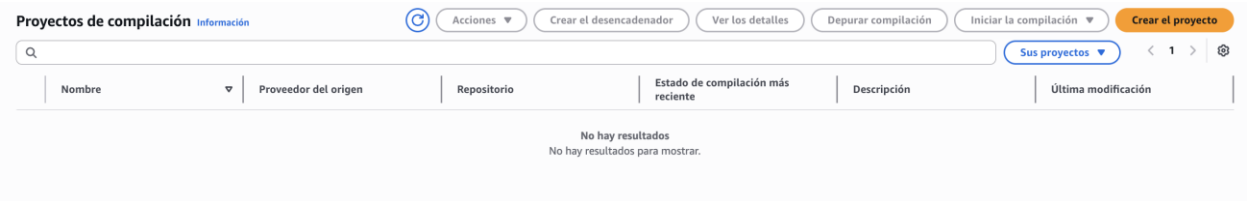

Amazon Web Services
CodePipeline
Source
Build
Test
Deploy
Amazon Web Services
CodeBuild

Actividad 4 – Implementación de proyecto en AWS CodeBuild

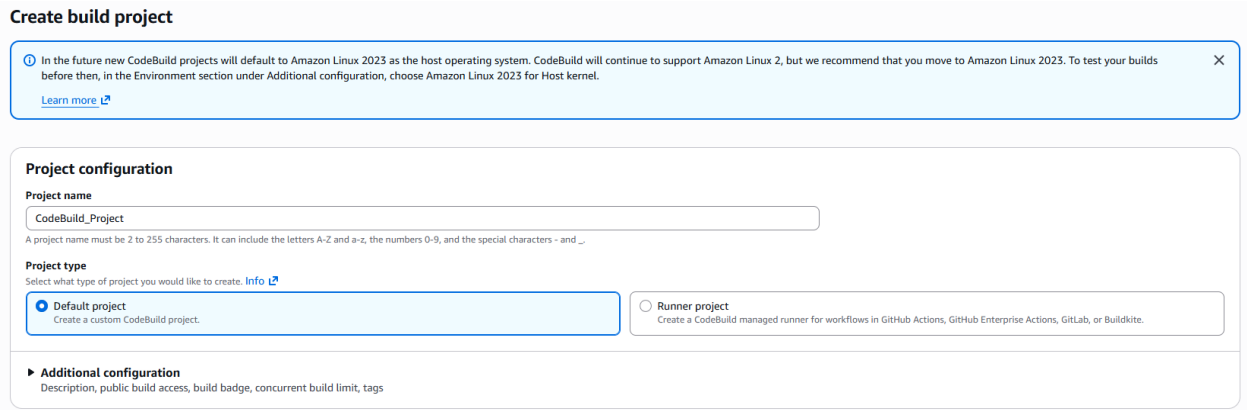
En la barra de búsqueda de la consola de AWS busque y seleccione **CodeBuild**.



En la ventana principal de CodeBuild de clic en el botón ***"Crear el Proyecto"***



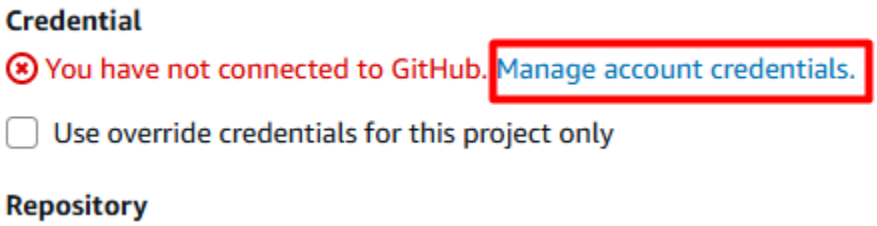
Asigne un nombre y descripción para el proyecto



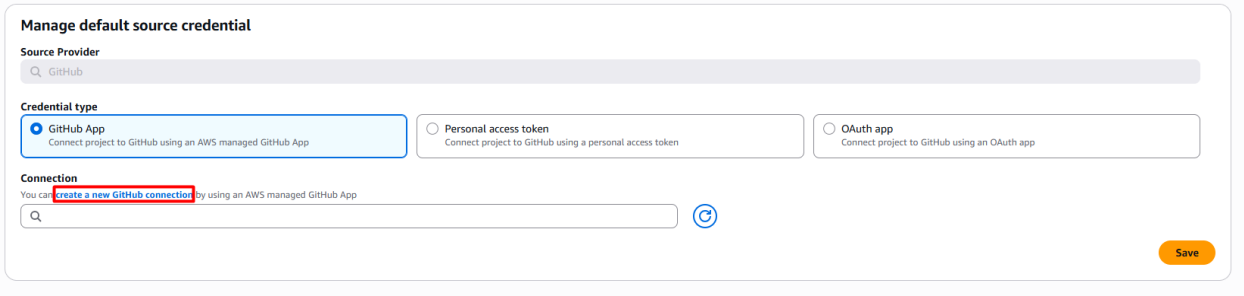
Agregue el origen del código de su aplicación y el repositorio, para este caso se utilizará como origen *GitHub* que es donde se almacenó con anterioridad el código de la aplicación de prueba.

Antes de seleccionar su repositorio, deberá configurar la conexión con su cuenta de GitHub, para esto selecciona la opción "*Administre las credenciales de la cuenta*"

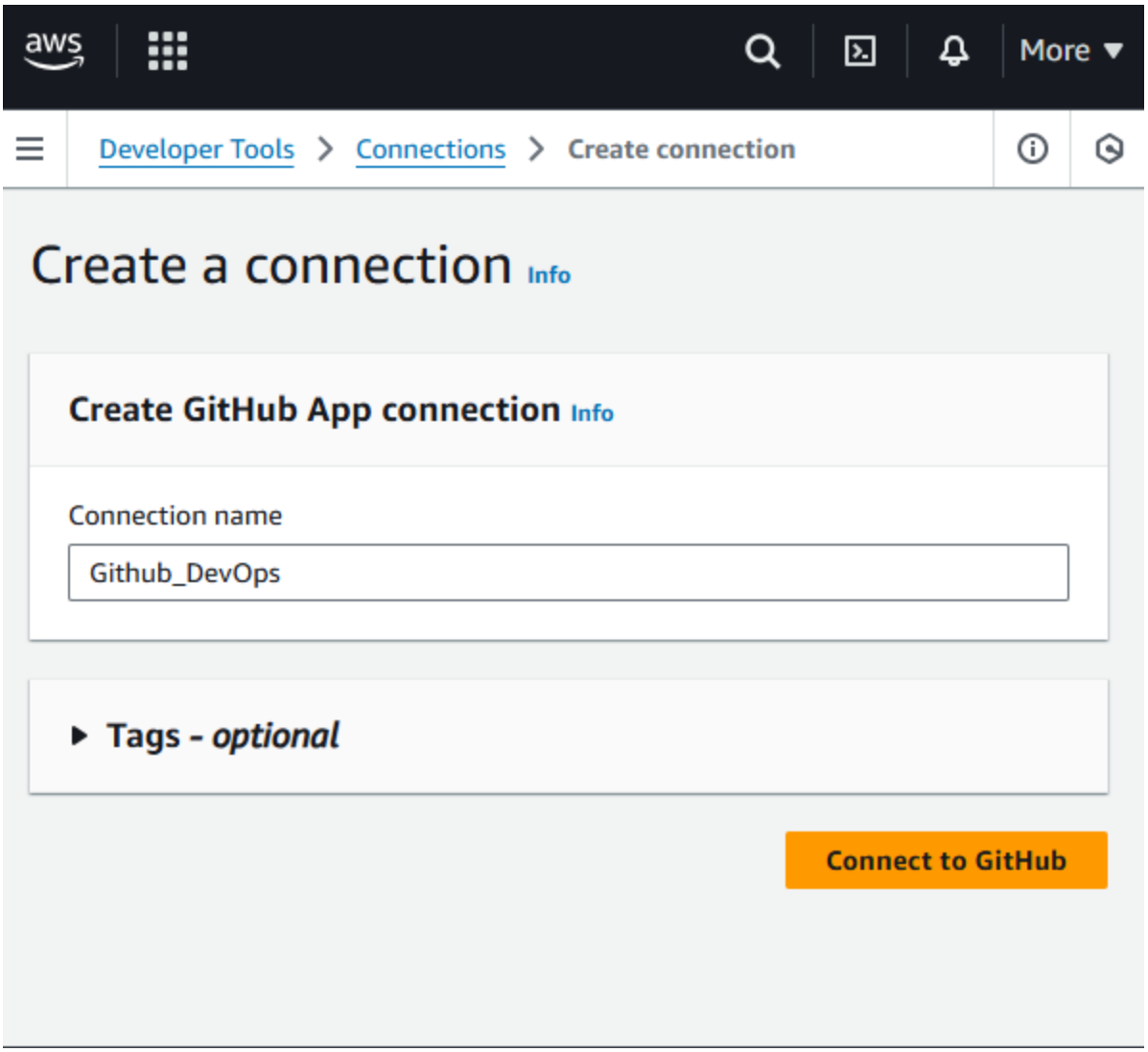




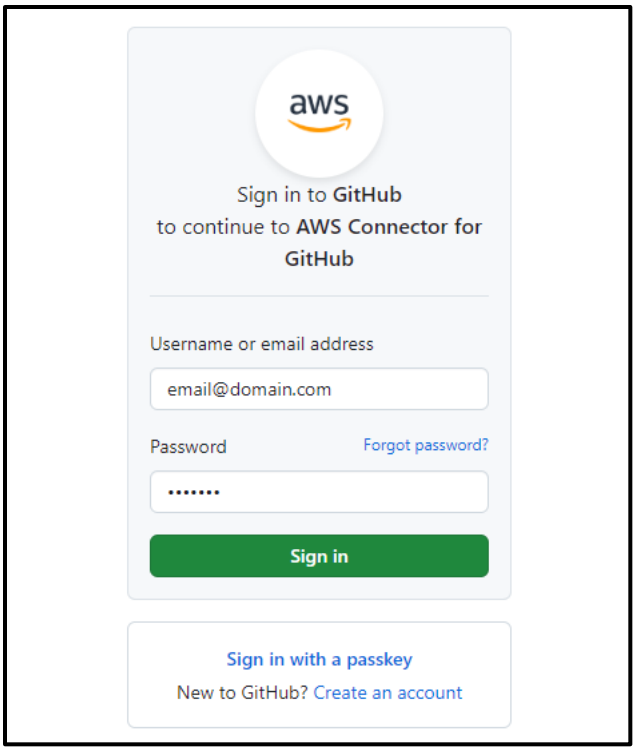
En la nueva ventana de configuración seleccione la opción "*Aplicación GitHub*" y de clic en "*Crear una nueva conexión de GitHub*"



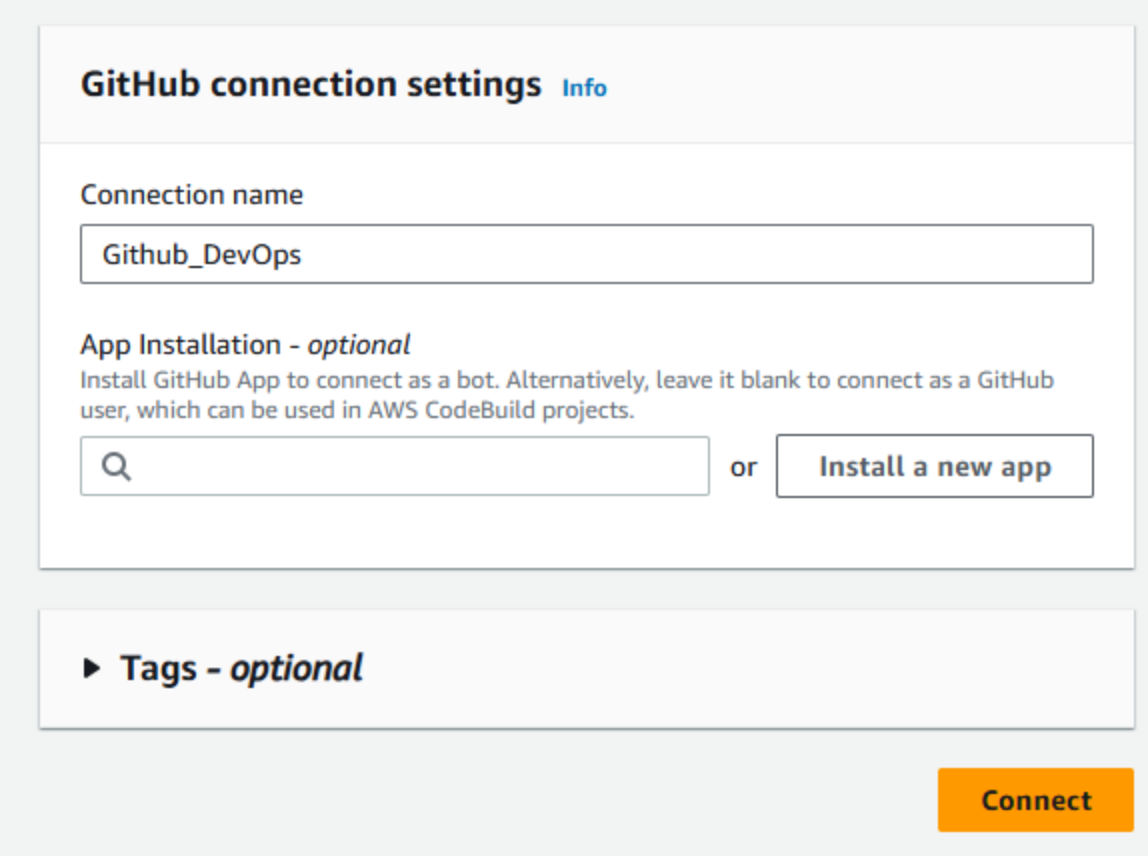
Se abrirá una nueva ventana emergente donde deberá asignar el nombre para la nueva conexión (para este caso GitHub Devops) y dar clic en el botón "*Conectarse a GitHub*".



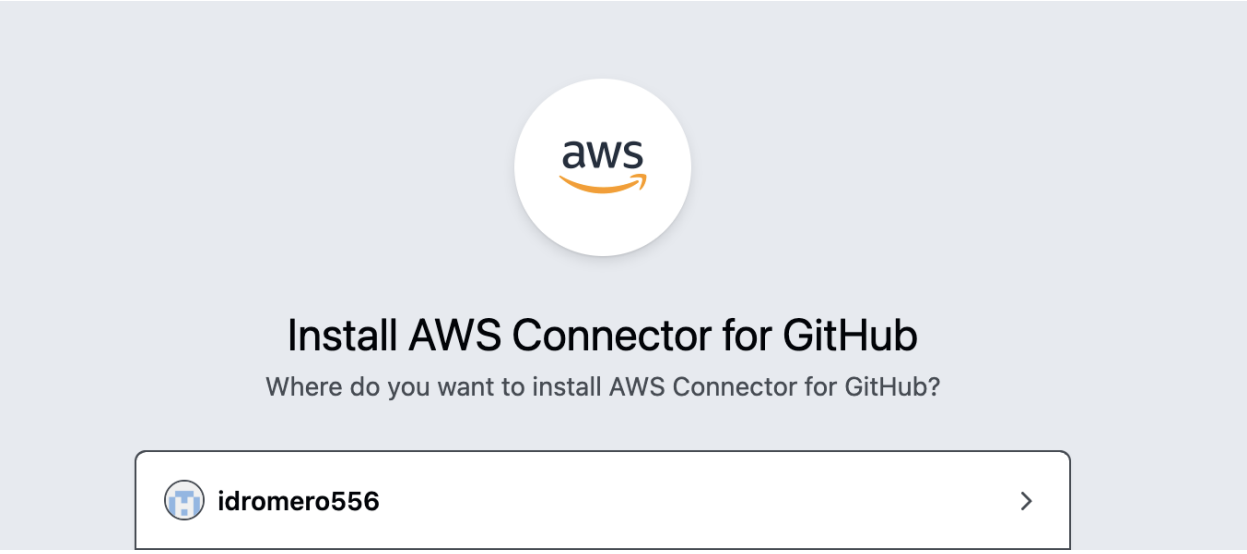
Se abrirá de nuevo una ventana emergente la cual le pedirá que inicie sesión en su cuenta de GitHub.



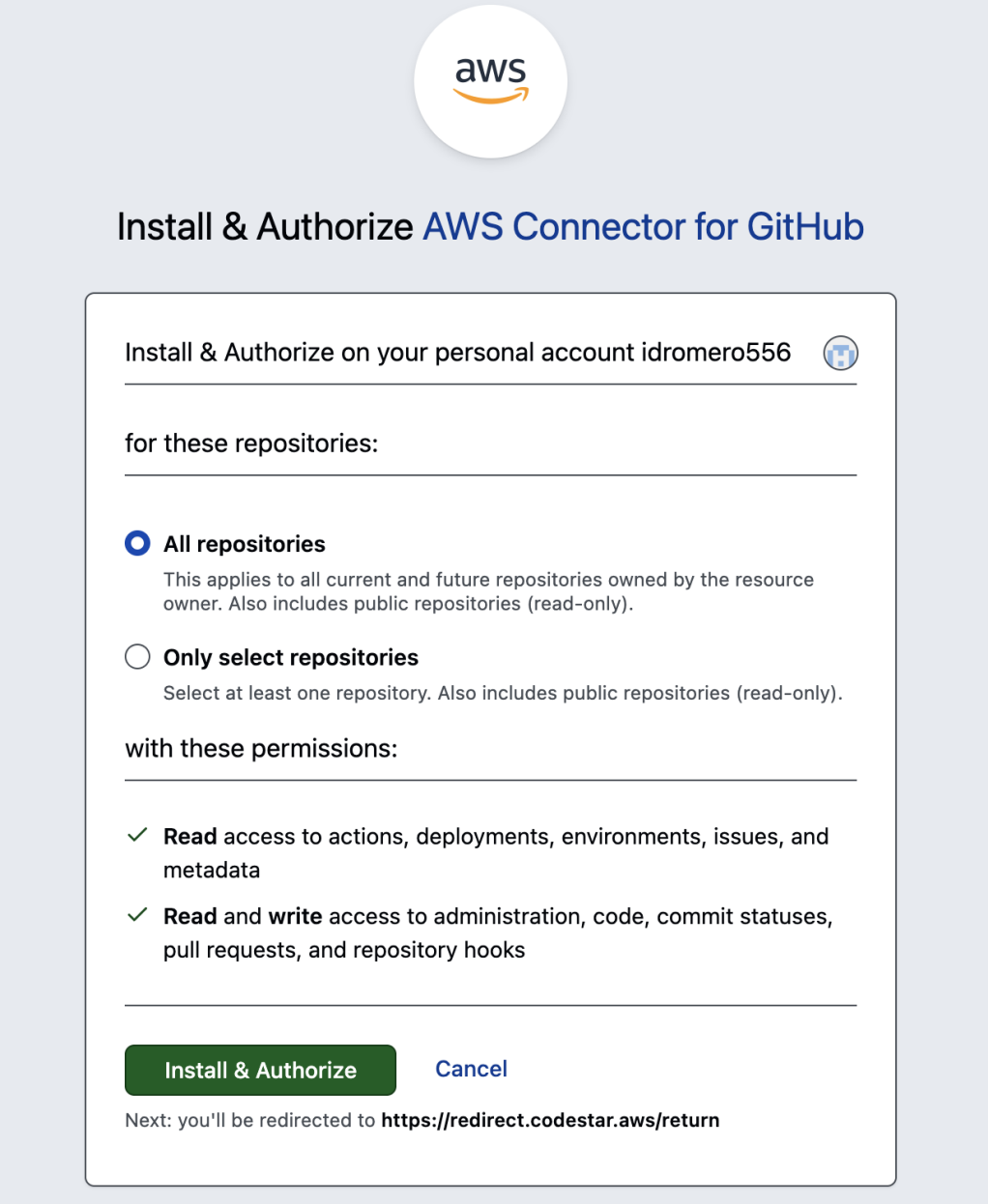
De clic en Instalar una nueva aplicación.



Seleccione donde desea instalar el conector.



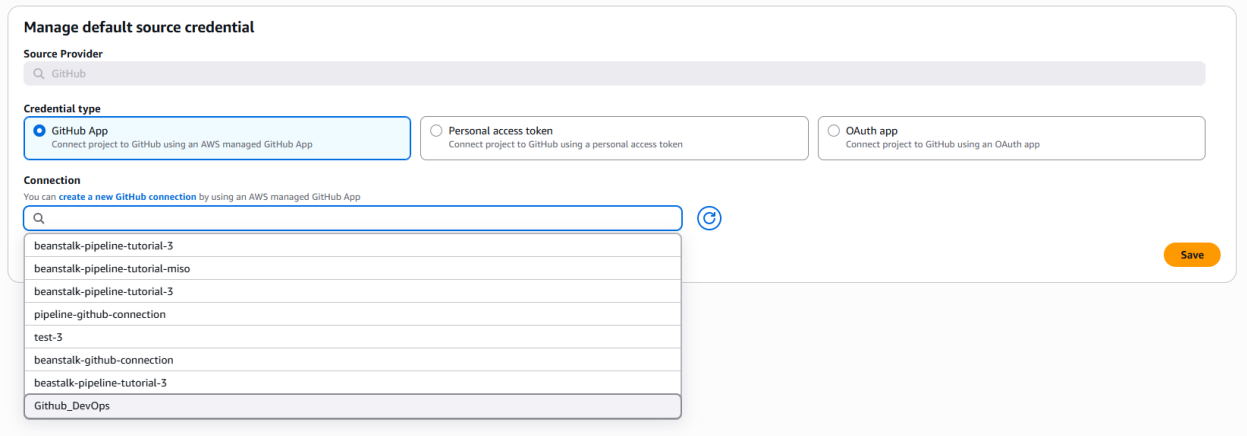
Puede seleccionar la conexión a un solo repositorio o a todos:

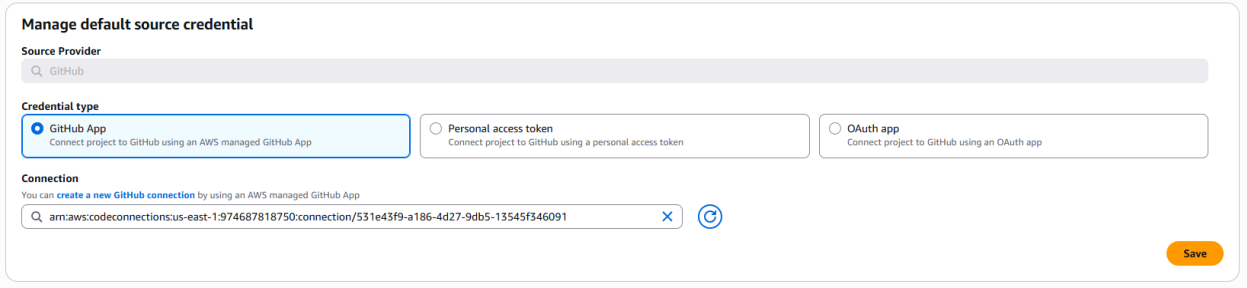


Luego de dar clic en Install & Authorize será redirigido a la consola de AWS nuevamente. De clic en Conectar



En la ventana de administración de credenciales de origen ya podrá seleccionar la nueva conexión con GitHub que creo en el paso anterior, selecciónela y de clic en "*Guardar*".



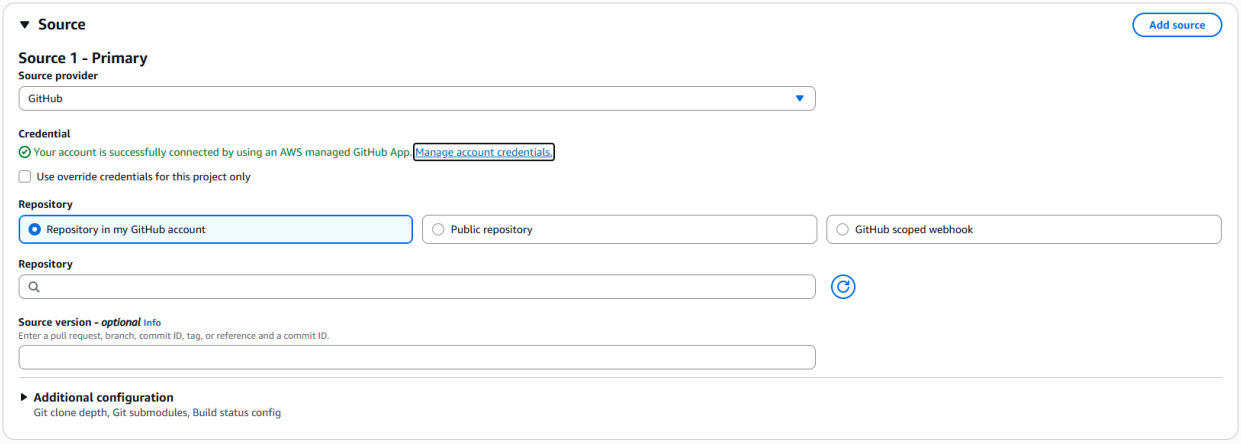


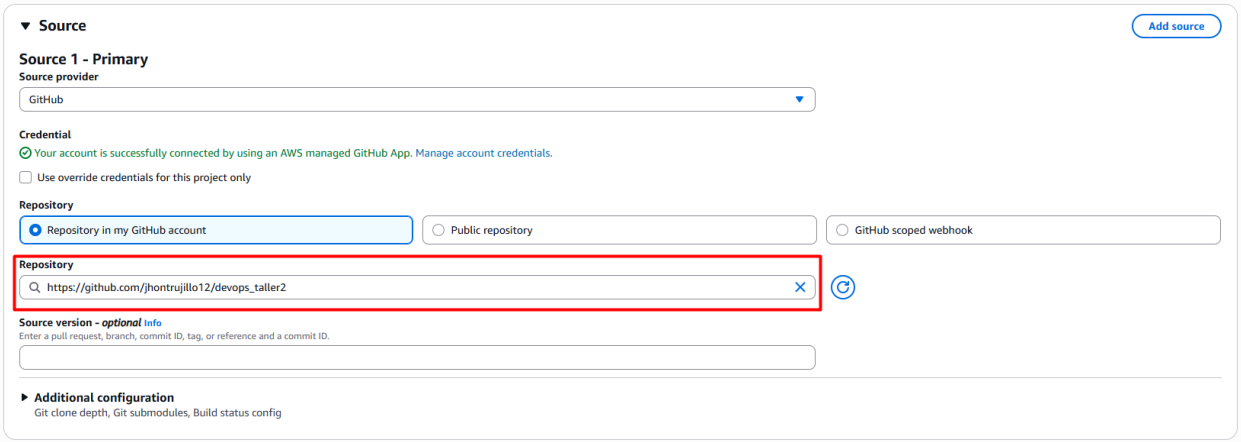
Regresará a la ventana de configuración del proyecto de CodeBuild. Seleccione el nombre de su repositorio y la versión de origen.

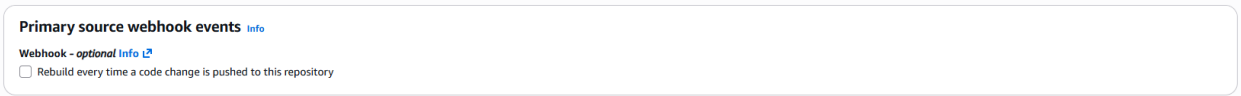
Puede seleccionar varios tipos de referencia de la versión del código que desea compilar y testear, dentro de las opciones está:

- *Ramificación:* Se podrá seleccionar la rama que desee desde el repositorio y sobre esta se realizará el testeo.
- *Etiqueta de Git:* Si puede referenciar una rama no permutada que cuente con un tag.
- *ID de confirmación:* Se podrá referenciar por cada uno de los commit que se hayan realizado sobre el repositorio y hacer test sobre cada uno de ellos.

Para este caso se dejará el valor en blanco lo cual tomará por defecto la rama master del repositorio.







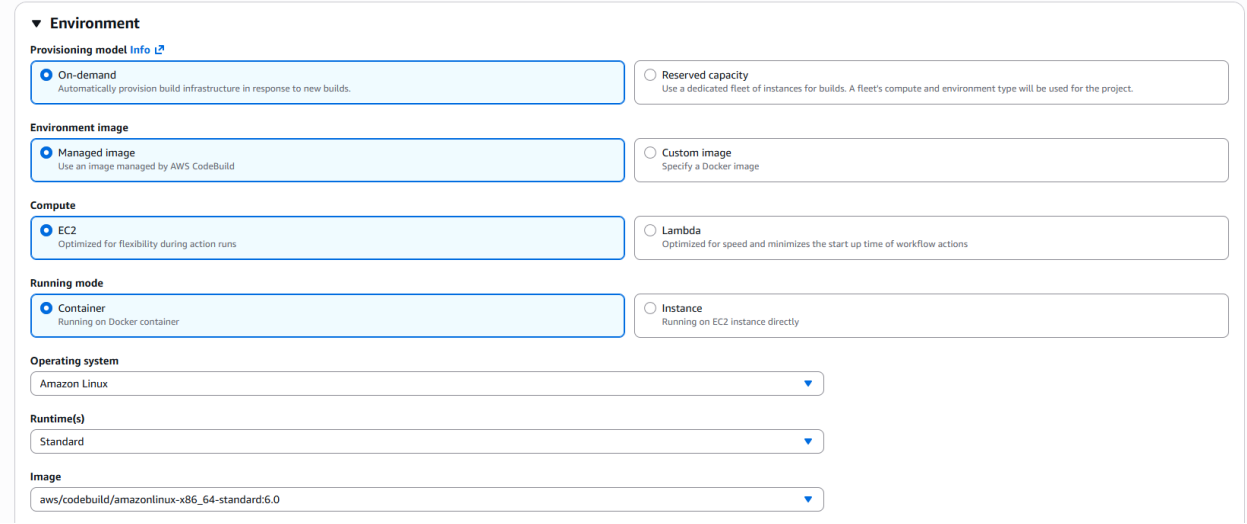
Para efectos de este tutorial deseleccione el Webhook.

A continuación, debe configurar el entorno, donde debe especificar el modelo de aprovisionamiento, para este caso será *Bajo demanda* lo cual permitirá que se aprovisione nueva infraestructura de compilación después de cada compilación.

Seleccione la imagen del entorno, entre estas puede utilizar una imagen administrada por AWS o una imagen de Docker personalizada. Para este caso se seleccionará una imagen administrada por AWS con sistema operativo Ubuntu.

Deje los valores de *Tiempo de ejecución* e *Imagen* como en la siguiente gráfica. Es recomendable que seleccione la versión de la imagen más actualizada.

Por otro lado, debe seleccionar o crear un *Rol de servicio* el cual permitirá que CodeBuild tenga acceso a los recursos necesarios. Para este caso, se creará un nuevo rol con el nombre por defecto.



En la siguiente sección encontrará la configuración de *BuildSpec* (*Especificación de compilación).*

BuildSpec es un archivo con formato *YAML (.yml)* el cual contiene un grupo de comandos de compilación que utiliza CodeBuild para ejecutar las compilaciones.

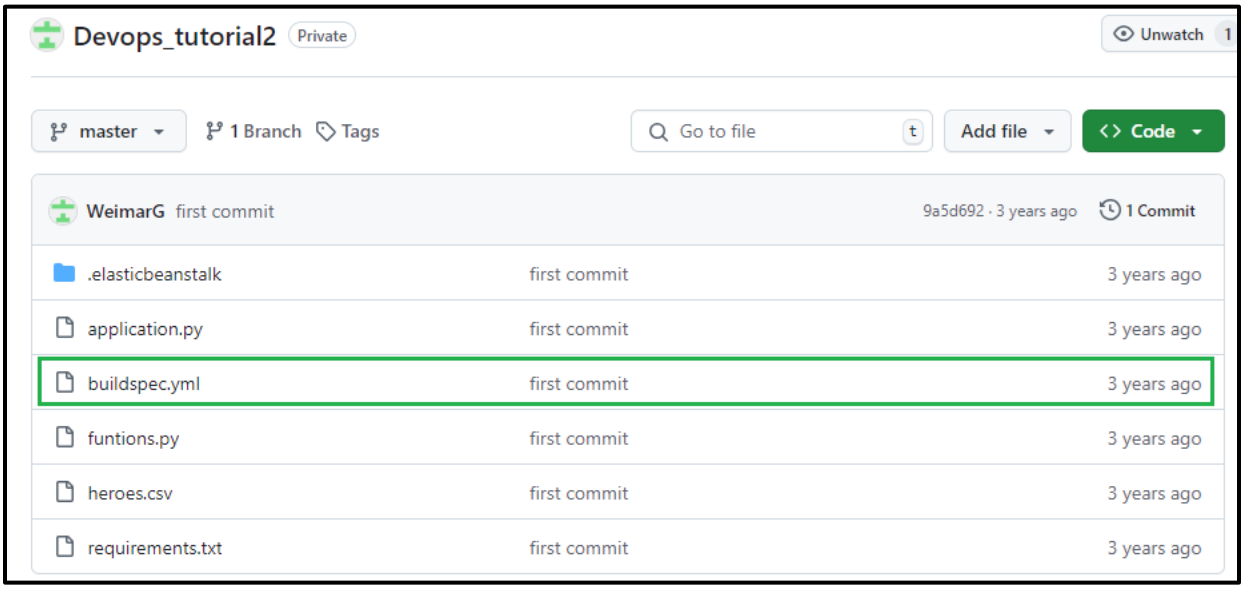
El archivo *BuildSpec.yml* puede estar dentro del código fuente o puede especificarlo en el momento de la creación del proyecto.

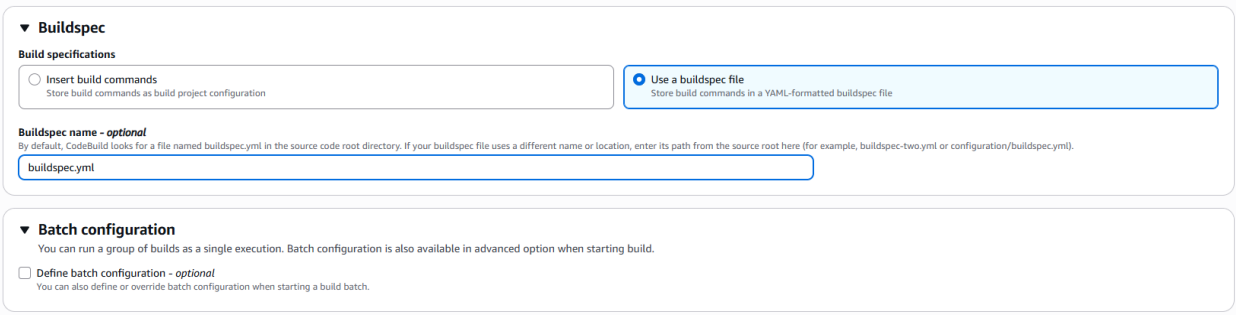
Se recomienda que el archivo tenga el nombre *buildspec.yml* en caso de que requiera usar otro nombre para diferenciar varias compilaciones, debe especificar este nombre en el momento que cree el proyecto de CodeBuild.

Para obtener más información de la construcción y sintaxis de BuildSpec diríjase al siguiente enlace:

https://docs.aws.amazon.com/es_es/codebuild/latest/userguide/build-spec-ref.html

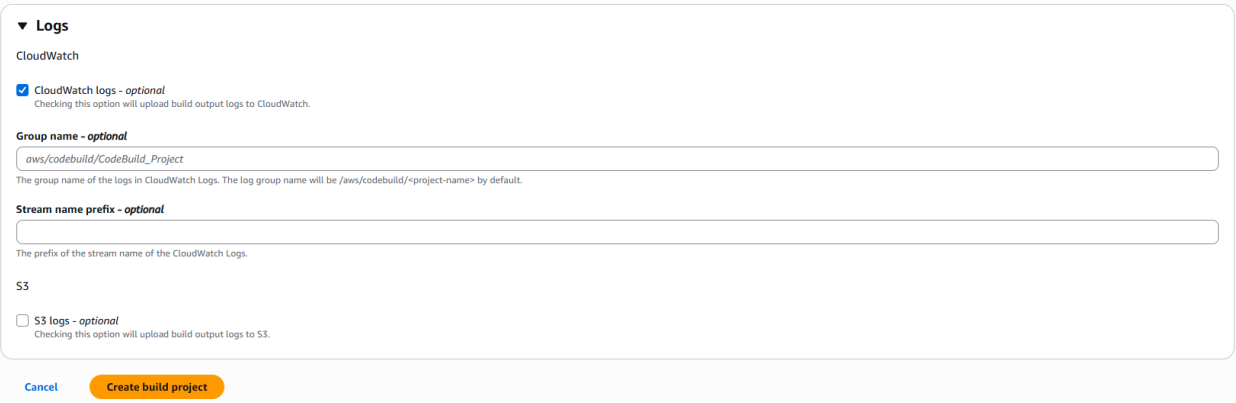
Para este ejemplo, se especificará que el archivo de compilación está dentro del código fuente.



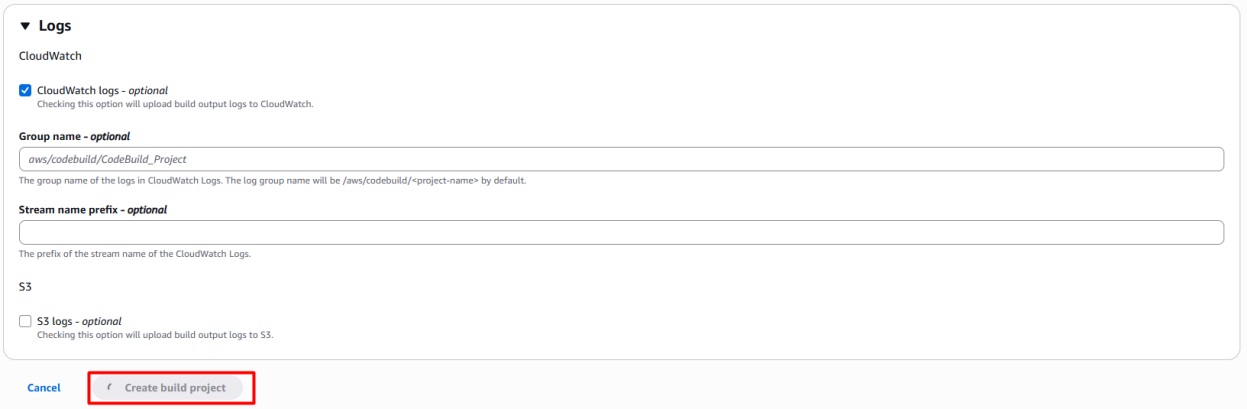


Por último, debe definir como quiere hacer el registro de logs y donde almacenarlos.

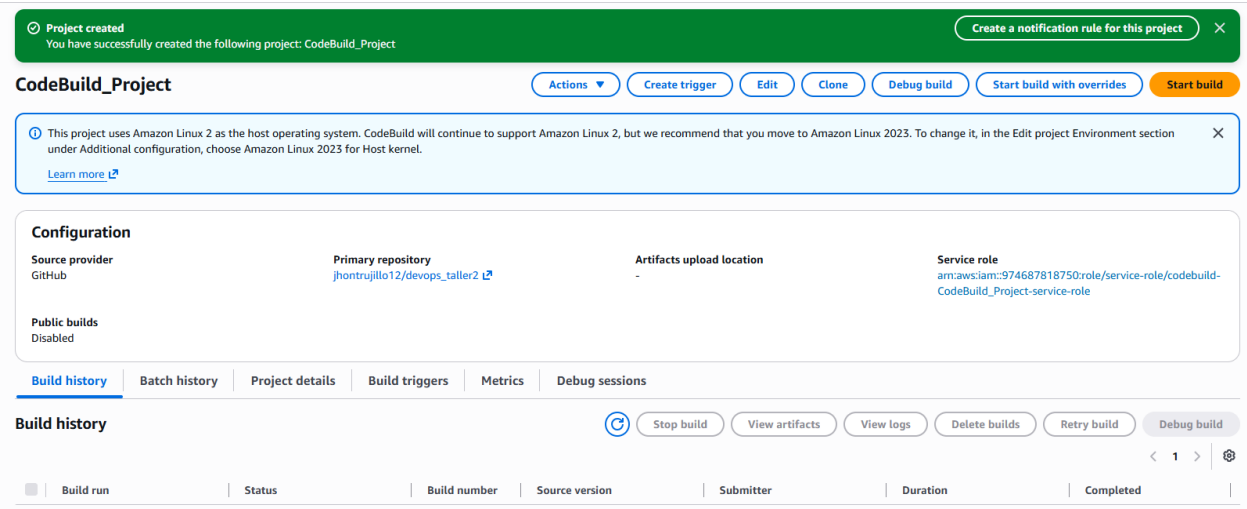
En este caso se hará el registro de los logs en CloudWatch.



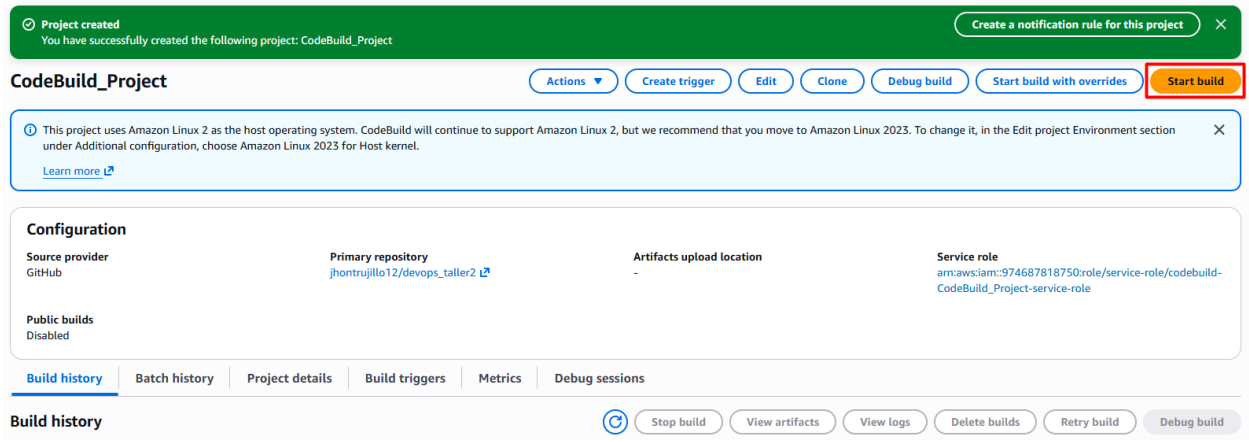
De clic en el botón “*Crear el proyecto de compilación*”



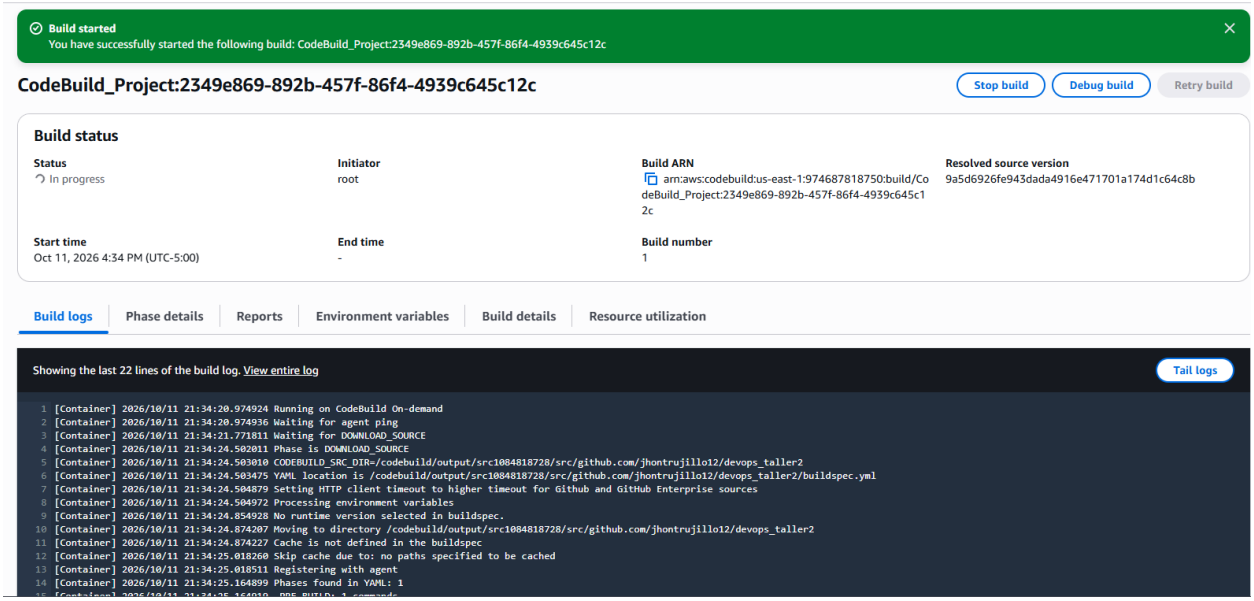
En esta parte estará construido el proyecto de CodeBuild y podrá iniciar con los Test de compilación.



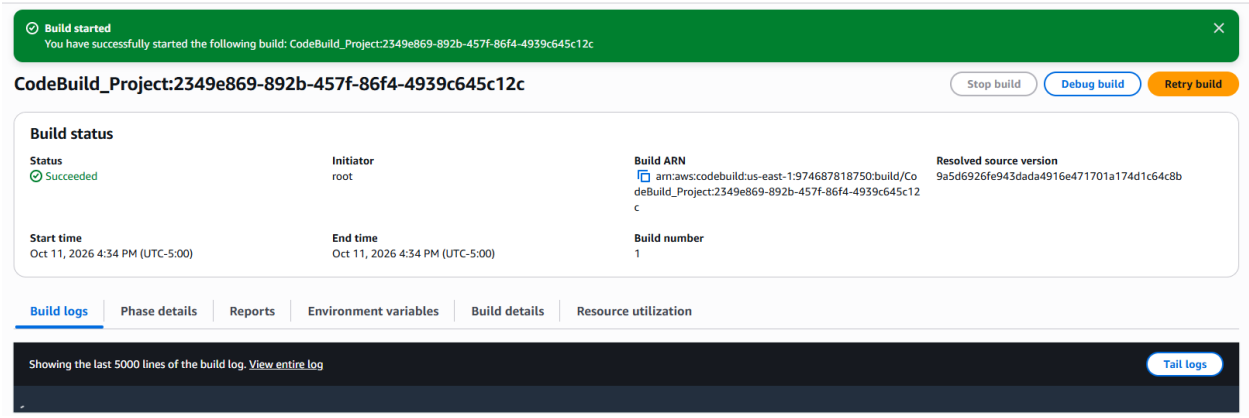
A continuación, de clic en el botón “***Iniciar la compilación***”, para iniciar las pruebas.



Podrá ver el estado de la compilación mientras se ejecuta.



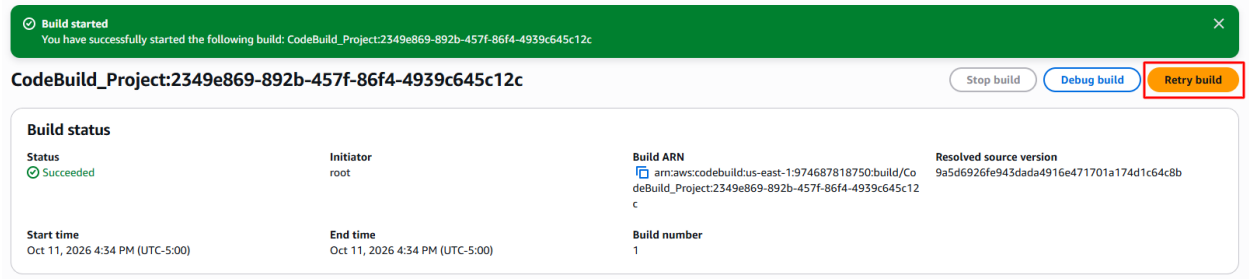
Si la compilación finalizó correctamente vera algo similar a la siguiente imagen en la sección Registros de compilación:



Explore las diferentes secciones del detalle de una compilación, esto le puede servir más adelante para hacer debug de la ejecución de una compilación y de posibles errores:

- Registros de compilación
- Detalles de la fase
- Informes
- Variables de entorno
- Detalles de la compilación
- Utilización de recursos

Si necesita volver a ejecutar una compilación puede usar la opción de “Volver a internar realizar una compilación”.



Actividad 5 – Actividades adicionales y consideraciones

A partir de este punto puede proceder a:

- Agregar cambios a los comandos en el archivo buildspec.yml en el repositorio en GitHub. Un ejemplo de un cambio puede ser agregar por ejemplo el comando de *echo* que imprima un texto durante la fase de pre_build en una compilación. Más información aquí.
- Después de los cambios realizados en el repositorio volver a ejecutar manualmente nuevas compilaciones por cada cambio realizado y validar que los cambios se visualizan en los registros de compilación.
- Agregar pruebas unitarias al proyecto.
- Realizar las configuraciones en el proyecto para que la ejecución de las pruebas se ejecute durante el proceso de build en una compilación y los resultados se puedan ver en los registros de la compilación.

- Configurar para que después de la ejecución de las pruebas se guarde el repositorio raíz en S3 como el artefacto que luego será desplegado a un ambiente. Con esto se guarda trazabilidad del resultado del artefacto generado en cada compilación.

Una vez terminadas las pruebas recuerde:

- Borrar el repositorio de GitHub
- Eliminar todos los recursos creados a lo largo de este tutorial

Referencias

[1] https://docs.aws.amazon.com/es_es/codecommit/latest/userguide/getting-started-cc.html

[2] https://docs.aws.amazon.com/es_es/codecommit/latest/userguide/setting-up.html

[3] https://docs.aws.amazon.com/es_es/codecommit/latest/userguide/setting-up-git-remote-codecommit.html

[4] https://docs.aws.amazon.com/es_es/cli/latest/userguide/install-cliv2-windows.html

[5] https://docs.aws.amazon.com/es_es/codebuild/latest/userguide/codebuild-user.pdf

[6] https://docs.aws.amazon.com/es_es/codebuild/latest/userguide/build-env-ref.html

[7] https://docs.aws.amazon.com/es_es/codebuild/latest/userguide/build-spec-ref.html

[8] https://docs.aws.amazon.com/es_es/codebuild/latest/userguide/concepts.html#concepts-how-it-works